6G-DALI Translation Engine

Developer Guide for Testbed Owners

Template version 1.0.0

How to run the template, implement your testbed, and comply with the common API.

Contents

1. Introduction

1.1 What the template gives you
1.2 Who does what

2. Quick start: run the template and open the API docs

2.1 Prerequisites
2.2 Get the code and start the engine
2.3 Open the API documentation
2.4 Try a full experiment cycle (optional)

3. Project structure: what to edit and what not to touch

3.1 What you edit
3.2 What you must not change

4. API endpoints

4.1 POST /6gdali/experiment/start
4.2 DELETE /6gdali/experiment/stop/{experiment_id}
4.3 Validation errors

5. Handling the request data

5.1 config: define and validate it
5.2 metadata: pass it to metadata generation
5.3 status_update_url and experiment state

6. Sending experiment states back

6.1 State values
6.2 How to send a state
6.3 Example: reporting the final state of the pipeline

7. Building responses: success and error

7.1 The standard response format
7.2 Returning an error from your code: APIException
7.3 The helpers in utils.py
7.4 Response codes

8. Request validation: a safety mechanism against LLM hallucination

8.1 How to validate config strictly

9. Running and stopping experiments

9.1 Why a safety net is needed
9.2 The max-duration watchdog

10. Implementation checklist

1. Introduction

The **Translation Engine** is a lightweight web service deployed alongside each virtual testbed. It exposes a standardized API through which the 6G-DALI Framework submits experiment-control requests. The Translation Engine maps these requests to the testbed-specific APIs and reports experiment status updates to the Framework.

All testbeds implement a common API contract, including standardized endpoints, request schemas, response structures, and experiment status values. This abstraction enables the Framework to orchestrate experiments across heterogeneous testbeds without requiring knowledge of their internal architectures or implementation details.

The Framework's request initiator is an **LLM-based agent** that interprets API responses to identify and correct request-related errors. Although the agent cannot resolve issues within the testbed's internal implementation, it can use structured error responses to diagnose invalid requests and retry operations with corrected parameters. Therefore, the Translation Engine must return precise, actionable error messages that conform to the standardized error-response format defined in this guide.

1.1 What the template gives you

- The two common endpoints (start and stop an experiment), already implemented.
- Validation of every request: experiment ID, metadata and status URL rules are built in.
- The standard response format for success and errors, and helpers to build it.
- A helper to send experiment states back to the Framework in the correct format.
- Swagger API documentation, served by the engine itself and usable offline.
- A simulated testbed, so the template runs on its own before you write any code.

1.2 Who does what

Translation Engine (common, provided)

- Exposes the start / stop endpoints
- Validates `exp_id`, `metadata`, `status_update_url` and `config`
- Returns every response in the standard format
- Formats status updates correctly
- Serves the Swagger documentation

Testbed owner (you implement)

- Defines the `config` fields of the testbed
- Translates `config` into the testbed's own API and starts / stops the experiment
- Passes `metadata` to the metadata generation solution
- Tracks experiments (the engine stores no state)
- Decides when the state changes and sends it back

> **Key principle**
>
> The Translation Engine **stores no experiment state**. It validates requests, calls your code and guarantees the format of what is sent back. Knowing which experiments exist, where they run and when they change state is the testbed owner's responsibility.

2. Quick start: run the template and open the API docs

The template can be executed locally without requiring an external testbed or additional infrastructure. It includes a simulated testbed implementation that enables users to explore and validate the standardized API endpoints and their behavior in a self-contained environment.

2.1 Prerequisites

- Python 3.9 or newer (`python3 --version`).
- Git.

2.2 Get the code and start the engine

```
git clone https://github.com/6G-DALI/adapter-translation-engine.git
cd adapter-translation-engine/translation-engine-template


python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

pip install -r requirements.txt
python translation_engine.py
```

When the engine is ready, the terminal shows:

```
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

2.3 Open the API documentation

Open **http://localhost:8000/docs** in a browser. The Swagger UI lists both endpoints, the request body with an example, every response code with an example, and all data models. Its files are bundled with the template, so it also works on a testbed network without Internet access.

URL	What it shows
`http://localhost:8000/docs`	Swagger UI (interactive documentation)
`http://localhost:8000/openapi.json`	The raw OpenAPI specification (for Postman, code generators, the LLM agent…)
`http://localhost:8000/`	Redirects to `/docs`

To use another port or interface, set environment variables: `PORT=8080 HOST=127.0.0.1 python translation_engine.py`.

2.4 Try a full experiment cycle (optional)

A small mock Framework is included to receive the status updates the engine sends back.

1. In a second terminal (same folder, virtual environment activated), run `python extras/mock_framework.py`. It listens on port 9000.
2. In Swagger, open **POST /6gdali/experiment/start**, click **Try it out**, keep the example body and click **Execute**. The response is `EXPERIMENT_START_ACCEPTED`.
3. After about one second, the mock Framework prints a `RUNNING` status update.
4. Open **DELETE /6gdali/experiment/stop/{experiment_id}**, enter `experiment-001` and click **Execute**. The mock Framework prints `COMPLETED`.

Try sending invalid bodies too (for example an `exp_id` starting with a capital letter, or a missing metadata key) to see the error responses.

3. Project structure: what to edit

File	Editable	Contents
`translation_engine.py`	**No**	Common endpoints, error handlers, Swagger setup, server start.
`models.py`	**Only the Configuration section**	Request / response models, experiment-ID rule, metadata model, response codes, states.
`utils.py`	**No**	Helpers: `success()`, `error()`, `APIException`, `send_status_update()`.
`testbed.py`	**Yes**	Your testbed logic: start, stop, metadata hand-off, experiment state.
`static/`	No	Swagger UI files for offline documentation.
`extras/mock_framework.py`	Optional	Mock Framework for local testing. Not deployed.
`requirements.txt`	Add only	Python dependencies. You may add the libraries your testbed needs.

3.1 What you edit

File	Item	What to do
`models.py`	`class Configuration`	Declare the fields of the `config` object your testbed accepts (section 5.1).
`models.py`	`EXAMPLE_CONFIG`	A valid example `config`, shown in Swagger "Try it out".
`testbed.py`	`TESTBED_NAME`	Your testbed's name, shown in the Swagger title.
deployment	`MAX_EXPERIMENT_DURATION`	Environment variable: maximum run time before the watchdog stops an experiment (section 9.2).
`testbed.py`	`_experiments`	Simulation only. Replace with your own way of tracking experiments.
`testbed.py`	`start_experiment()`	Translate `config` and start the experiment on your testbed.
`testbed.py`	`process_metadata()`	Pass `metadata` to your metadata generation solution.
`testbed.py`	`stop_experiment()`	Stop endpoint: return 404 for unknown experiments; it cancels the watchdog, then calls `stop_procedure()`.
`testbed.py`	`stop_procedure()`	Your stopping procedure, used by the stop endpoint and by the watchdog (section 9.2).

Every place to change in `testbed.py` is marked with a `TODO` comment. You may add new modules (an API client for your testbed, helpers, constants) and import them from `testbed.py`.

3.2 What you must not change

- Endpoint paths, HTTP methods and request bodies.
- The response format (`APIResponse`), the response codes (`ResponseCode`) and error types (`ErrorType`).
- The state values (`NewAdapterState`) and the status update format (`AdapterStateUpdate`).
- The experiment-ID rule and the `Metadata` model.
- The error handlers in `translation_engine.py`.

> **Why**
>
> The Framework relies on these being identical on every testbed. If the common contract needs to change, raise it with the 6G-DALI Framework maintainers instead of editing it locally.

4. API endpoints

Both endpoints are under the `/6gdali` prefix. Every response, success or error, uses the standard format described in section 7.

4.1 POST /6gdali/experiment/start

Starts an experiment. Example request body:

```
{
  "exp_id": "experiment-001",
  "metadata": {
    "title": "Energy Saving Small Area Two Layers Sectorized AppOFF",
    "description": "RAN dataset generated through the EURECOM Digital Twin testbed",
    "keywords": ["Energy", "RAN", "Digital_Twin"],
    "testbed_context": { "network_domain": "RAN" }
  },
  "status_update_url": "http://localhost:8003/6gdali/experiment/status",
  "config": {
    "scenario_name": "Energy Saving Small Area Two Layers Sectorized AppOFF - flexric",
    "frequency": { "interval": 5, "unit": "seconds" }
  }
}
```

Field	Type	Rules (checked by the engine)	Used for
`exp_id`	string	Required, not empty, first character a lowercase letter `a-z`.	Identifies the experiment in all later calls and status updates.
`metadata`	object	Fixed keys, all required, no other key allowed (section 5.2).	Metadata generation only. **Not** used to run the experiment.
`status_update_url`	string (URL)	Required, valid http(s) URL.	Where you send the experiment states.
`config`	object	Must match your `Configuration` model.	Running the experiment on your testbed.

What the engine does, in order:

1. Validates the whole body. If anything is wrong, it replies with an error and your code is not called.
2. Calls `testbed.start_experiment(exp_id, config, status_update_url)`.
3. If that succeeds, calls `testbed.process_metadata(exp_id, metadata)`.

4. Replies `200 EXPERIMENT_START_ACCEPTED`.

The reply only means the request was **accepted**. Progress (RUNNING, COMPLETED, FAILED) is reported later through status updates (section 6).

HTTP	Code	When
200	`EXPERIMENT_START_ACCEPTED`	Request accepted.
400	`INVALID_EXPERIMENT_ID`	`exp_id` missing, empty or not starting with a-z.
400	`INVALID_METADATA`	`metadata` has a missing, extra or invalid key.
400	`INVALID_CONFIGURATION`	Raised by your code: `config` is valid but your testbed cannot run it.
409	`EXPERIMENT_ALREADY_EXISTS`	Raised by your code: this experiment already exists.
422	`INVALID_REQUEST`	Any other validation problem (bad `config`, bad URL, missing field...).
500	`TESTBED_ERROR`	Raised by your code: the testbed failed or refused the request.
500	`INTERNAL_ERROR`	An unexpected exception in the engine or your code.
503	`TESTBED_UNAVAILABLE`	Raised by your code: the testbed is offline or unreachable.

4.2 DELETE /6gdali/experiment/stop/{experiment_id}

Stops an experiment. `experiment_id` is the `exp_id` used at start and follows the same rule (first character a-z). There is no request body.

The engine validates the ID, calls `testbed.stop_experiment(exp_id)` and replies `200 EXPERIMENT_STOP_ACCEPTED`. As for start, the final state is reported later through a status update.

HTTP	Code	When
200	`EXPERIMENT_STOP_ACCEPTED`	Request accepted.
400	`INVALID_EXPERIMENT_ID`	ID does not start with a-z.
404	`EXPERIMENT_NOT_FOUND`	Raised by your code: unknown experiment.
500	`TESTBED_ERROR` / `INTERNAL_ERROR`	The testbed failed / unexpected exception.

4.3 Validation errors

When a request is invalid, the engine replies **before** calling your code. `error.details` lists **every** problem found, each with `loc` (the path to the field) and `msg` (what is wrong), so the LLM agent can correct the whole body in one retry. Example:

```
{
  "result": "error",
  "code": "INVALID_METADATA",
  "message": "Invalid metadata. Fix the fields listed in error.details.",
  "data": null,
  "error": {
    "type": "validation_error",
    "details": [
      {"loc": ["body", "metadata", "testbed_context", "network_domain"],
       "msg": "Field required"},
      {"loc": ["body", "metadata", "testbed_context", "domain"],
       "msg": "Extra inputs are not permitted"}
    ]
  },
  "timestamp": "2026-10-14T09:26:41.769000+00:00"
}
```

If several parts of the body are wrong, the `code` is chosen by priority: `INVALID_EXPERIMENT_ID`, then `INVALID_METADATA`, then `INVALID_REQUEST`. `error.details` always contains all problems.

5. Handling the request data

5.1 config: define and validate it

The template only defines the `config` key. **You must define what goes inside it** as a Pydantic model in the Configuration section of `models.py`. The engine then validates every request against your model before your code runs: a missing field, a wrong type or an unknown key is rejected with `422 INVALID_REQUEST`, listing the fields to fix.

Example matching the `config` shown in section 4.1:

```
class Frequency(BaseModel):
    model_config = ConfigDict(extra="forbid")
    interval: int = Field(..., gt=0, description="Sampling interval.")
    unit: str = Field(..., description="Unit of interval, e.g. 'seconds'.")

class Configuration(BaseModel):
    """Testbed-specific experiment configuration."""
    # Reject any key that is not declared as a field below.
    model_config = ConfigDict(extra="forbid")

    scenario_name: str = Field(..., description="Scenario to deploy.")
    frequency: Frequency

EXAMPLE_CONFIG = {
    "scenario_name": "Energy Saving Small Area Two Layers Sectorized AppOFF - flexric",
    "frequency": {"interval": 5, "unit": "seconds"},
}
```

- Keep `extra="forbid"`: a misspelled key is then rejected instead of silently ignored.
- Give every field a `description`: it appears in Swagger and helps the LLM agent build correct requests.
- Put type and range checks in the model (`gt=0`, enums, lengths). Put checks that need the testbed (does this scenario exist?) in `start_experiment()` and raise `INVALID_CONFIGURATION`.
- As shipped, `Configuration` has no fields, so only an empty object `{}` is accepted.

In `start_experiment()`, `config` is an instance of your model: read it as `config.scenario_name`, `config.frequency.interval`, and so on, and translate it into your testbed's request.

5.2 metadata: pass it to metadata generation

The metadata keys are **fixed for all testbeds** and validated by the engine (`models.Metadata`). All keys are required and no other key is accepted.

Key	Type	Rule
`title`	string	Not empty.
`description`	string	Not empty.
`keywords`	list of strings	At least one keyword; none empty.
`testbed_context`	object	Contains exactly `network_domain` (string, not empty), e.g. `"RAN"`.

Metadata is **not used to run the experiment**. The engine passes it to `process_metadata(exp_id, metadata)`, where you hand it to your testbed's metadata generation solution. It is called only after `start_experiment()` succeeded, so rejected experiments never produce metadata.

```
async def process_metadata(exp_id: str, metadata: Metadata) -> None:
    # metadata.title, metadata.keywords, metadata.testbed_context.network_domain ...
    async with httpx.AsyncClient() as client:
        await client.post(f"{METADATA_SERVICE_URL}/experiments/{exp_id}",
                          json=metadata.model_dump())
```

If `process_metadata()` raises, the Framework receives an error although the experiment has already started. Handle such failures inside the function (log, retry, or stop the experiment and report FAILED).

5.3 status_update_url and experiment state

`start_experiment()` receives `status_update_url`: the URL where states for this experiment must be sent. The stop request does not carry it, so **store it** together with whatever else you need (local job ID, start time...).

The engine keeps no state. You decide how to track experiments: in memory, a file, a database, or by querying your testbed's own API. The template's `_experiments` dictionary is a simulation placeholder; it is lost when the engine restarts. Your tracking is also what lets you return `409 EXPERIMENT_ALREADY_EXISTS` on start and `404 EXPERIMENT_NOT_FOUND` on stop.

6. Sending experiment states back

6.1 State values

State	Meaning
`PENDING`	Accepted, not running yet (queued, deploying).
`RUNNING`	The experiment is running on the testbed.
`COMPLETED`	The whole pipeline succeeded (see the rule below). Final.
`FAILED`	Any step failed. Final.

Only these four values exist. If your testbed has more detailed states (QUEUED, DEPLOYING, UPLOADING...), map each of them to one of the four. Every started experiment must end with exactly one final state: `COMPLETED` or `FAILED`. Nothing is sent after a final state.

> **Rule: when to send COMPLETED**
>
> Send `COMPLETED` **only** when **all** of the following succeeded:
>
> - the experiment ran successfully;
> - the **dataset and its metadata** were generated;
> - the files were **uploaded by the DS connector**.
>
> **Any failure in between** (experiment run, dataset generation, metadata generation or upload) is a failure of the experiment and must be reported as `FAILED`.

Typical sequence: `RUNNING` when the testbed starts the experiment, then, when the whole pipeline has finished, either `COMPLETED` or `FAILED`. A stop request does not mean `COMPLETED` by itself: report `COMPLETED` only once the steps above have succeeded for the data collected so far, otherwise `FAILED`.

6.2 How to send a state

Use the `send_status_update()` helper from `utils.py`. It builds the body in the correct format, POSTs it to the Framework, and returns `True` if the Framework accepted it, `False` otherwise (failures are logged, never raised).

```
from models import NewAdapterState
from utils import send_status_update

await send_status_update(exp_id, NewAdapterState.RUNNING, status_update_url)
```

Body sent to `status_update_url`:

```
{
  "exp_id": "experiment-001",
  "new_status": "RUNNING",
  "timestamp": "2026-10-14T09:26:42.115000+00:00"
}
```

- Always pass a `NewAdapterState` value, never a plain string.
- Never build or POST the body yourself: the helper guarantees the format.
- `start_experiment()` and `stop_experiment()` must return within a few seconds. Run long work (the experiment, dataset generation, upload) in the background with `asyncio.create_task(...)` and send the state when it finishes.

6.3 Example: reporting the final state of the pipeline

A background task started from `start_experiment()`. The function names inside `try` are placeholders for your own pipeline steps.

```
async def run_pipeline(exp_id: str, status_update_url: str) -> None:
    try:
        await run_experiment_on_testbed(exp_id)        # experiment runs
        await send_status_update(exp_id, NewAdapterState.RUNNING, status_update_url)
        await wait_until_experiment_finished(exp_id)
        await generate_dataset_and_metadata(exp_id)    # dataset + metadata
        await upload_with_ds_connector(exp_id)         # DS connector upload
    except Exception:
        logger.exception("Pipeline failed for '%s'", exp_id)
        await send_status_update(exp_id, NewAdapterState.FAILED, status_update_url)
        return
    # Reached only if EVERY step above succeeded.
    await send_status_update(exp_id, NewAdapterState.COMPLETED, status_update_url)


async def start_experiment(exp_id, config, status_update_url):
    ...  # checks, translation, store status_update_url
    asyncio.create_task(run_pipeline(exp_id, status_update_url))
```

7. Building responses: success and error

7.1 The standard response format

Every endpoint returns this structure (`APIResponse`), for success and for errors:

Field	Content
`result`	`"success"` or `"error"`.
`code`	A value from `ResponseCode` (table in 7.4).
`message`	Human-readable explanation. Read by the LLM agent: be specific.
`data`	Payload of a success (e.g. `{"experiment_id": "experiment-001"}`), otherwise `null`.
`error`	`null` on success; on error `{"type": <ErrorType>, "details": <anything useful>}`.
`timestamp`	Time of the response, ISO-8601, UTC.

The success responses of the two endpoints are already produced by `translation_engine.py`. In `testbed.py` you only need to **raise errors**.

7.2 Returning an error from your code: APIException

Raise `APIException` anywhere in `testbed.py`. The engine turns it into the standard error response with the HTTP status you give.

```
from models import ErrorType, ResponseCode
from utils import APIException

raise APIException(
    400,                                    # HTTP status
    ResponseCode.INVALID_CONFIGURATION,     # code
    "Scenario 'foo' does not exist on this testbed.",   # message
    ErrorType.VALIDATION_ERROR,             # error.type
    details={"field": "config.scenario_name",           # error.details (optional)
             "allowed": ["scenario-a", "scenario-b"]},
)
```

Produces:

```
{
  "result": "error",
  "code": "INVALID_CONFIGURATION",
  "message": "Scenario 'foo' does not exist on this testbed.",
  "data": null,
  "error": {
    "type": "validation_error",
    "details": {"field": "config.scenario_name",
                "allowed": ["scenario-a", "scenario-b"]}
  },
  "timestamp": "2026-10-14T09:30:00.000000+00:00"
}
```

Typical cases:

```
# Testbed offline or not answering
try:
    async with httpx.AsyncClient(timeout=10) as client:
        resp = await client.post(f"{TESTBED_URL}/jobs", json=local_request)
except httpx.TransportError:
    raise APIException(503, ResponseCode.TESTBED_UNAVAILABLE,
                       "The testbed is currently unavailable.",
                       ErrorType.TESTBED_ERROR)

# Testbed answered with an error
if resp.status_code >= 400:
    raise APIException(500, ResponseCode.TESTBED_ERROR,
                       "Testbed refused the experiment.",
                       ErrorType.TESTBED_ERROR, details=resp.text)

# Unknown experiment on stop
raise APIException(404, ResponseCode.EXPERIMENT_NOT_FOUND,
                   f"Experiment '{exp_id}' not found.", ErrorType.NOT_FOUND)
```

> **Do not**
>
> Do not use FastAPI's `HTTPException`: it wraps the body in `{"detail": ...}` and breaks the standard format.
>
> Do not return error dictionaries yourself, and do not invent new codes. If no code fits exactly, use the closest one and put the specifics in `details`.
>
> Never put secrets (tokens, passwords) in `message` or `details`.

Any other exception that escapes your code is returned as `500 INTERNAL_ERROR` and logged with its traceback. Prefer catching expected failures and raising the right `APIException`, so the agent receives a meaningful code.

7.3 The helpers in utils.py

Helper	Purpose
`APIException(http_status, code, message, err_type, details=None)`	Raise to return an error response. The one you use in `testbed.py`.
`send_status_update(exp_id, new_status, status_update_url)`	Send a state to the Framework (section 6.2). Async; returns True/False.
`success(code, message, data=None)`	Builds a success response dict. Used by the engine.

Helper	Purpose
`error(code, message, err_type, details=None)`	Builds an error response dict. Used by the engine and `APIException`.

7.4 Response codes

Code	HTTP	error.type	Raised by	Meaning
`EXPERIMENT_START_ACCEPTED`	200	-	Engine	Start request accepted.
`EXPERIMENT_STOP_ACCEPTED`	200	-	Engine	Stop request accepted.
`INVALID_EXPERIMENT_ID`	400	validation_error	Engine	ID missing, empty or not starting with a-z.
`INVALID_METADATA`	400	validation_error	Engine	Metadata key missing, extra or invalid.
`INVALID_CONFIGURATION`	400	validation_error	Testbed	Valid config the testbed cannot run.
`EXPERIMENT_NOT_FOUND`	404	not_found	Testbed	Stop for an unknown experiment.
`ENDPOINT_NOT_FOUND`	404	not_found	Engine	Unknown URL.
`METHOD_NOT_ALLOWED`	405	validation_error	Engine	Wrong HTTP method.
`EXPERIMENT_ALREADY_EXISTS`	409	conflict	Testbed	Start for an existing experiment.
`INVALID_REQUEST`	422	validation_error	Engine	Any other validation problem.
`TESTBED_ERROR`	500	testbed_error	Testbed	The testbed failed or refused.
`INTERNAL_ERROR`	500	adapter_error	Engine	Unexpected exception.
`TESTBED_UNAVAILABLE`	503	testbed_error	Testbed	Testbed offline or unreachable.

8. Request validation: a safety mechanism against LLM hallucination

Experiment requests are produced by the intent translation solution, an LLM-based agent. LLMs can **hallucinate**: they may invent a configuration key, misspell a scenario name, or propose a parameter outside the range the testbed supports (a negative bandwidth, an interval of zero, an unrealistic number of nodes). Forwarded to the testbed, such values can break testbed-side components or leave them in an inconsistent state.

Pydantic validation is the Translation Engine's safety mechanism against this. Every request is checked at the entry point, before any testbed code runs. An invalid request is rejected immediately with a precise error (section 4.3), which the agent uses to correct its request. Incorrect configurations and out-of-range parameters therefore never reach the testbed.

What is validated	Where and by whom
`exp_id`, `metadata`, `status_update_url`	Common models in `models.py`. Already implemented by the template.
`config` (fields, types, ranges)	Your `Configuration` model. **You must define it strictly** (8.1).
Rules that need the testbed (e.g. is this scenario available?)	`start_experiment()`: raise `INVALID_CONFIGURATION` with details on what is allowed.

8.1 How to validate config strictly

- Declare every field your testbed accepts and keep `extra="forbid"`, so unknown or misspelled keys are rejected.
- Give every numeric field a range (`gt`, `ge`, `lt`, `le`) matching what the testbed can really handle.
- Use enums (or `Literal`) for fixed choices such as scenario names or units, so only known values pass.
- Limit strings and lists with `min_length` / `max_length`.
- Write a `description` on every field, including its unit and range: the agent reads it in the API documentation.

Example (field names and limits are illustrative):

```
class ScenarioName(str, Enum):
    ENERGY_SAVING = "energy-saving-small-area"
    MOBILITY = "mobility-urban"

class Configuration(BaseModel):
    model_config = ConfigDict(extra="forbid")

    scenario_name: ScenarioName = Field(..., description="Scenario to deploy.")
    bandwidth_mhz: int = Field(..., ge=5, le=100,
                               description="Channel bandwidth in MHz (5-100).")
    num_ues: int = Field(..., ge=1, le=50,
                         description="Number of simulated UEs (1-50).")
```

> **Rule of thumb**
>
> If a value would be unsafe or meaningless for your testbed, your model must reject it. Rejecting a request costs one retry for the agent; a broken testbed component costs much more.

9. Running and stopping experiments

The intent translation solution **starts and stops** every experiment. There is no duration in the `config`, and the testbed does not stop experiments by itself: an experiment runs until the stop endpoint is called. The testbed owner implements what stopping means for the testbed in `stop_procedure()` in `testbed.py`.

9.1 Why a safety net is needed

The testbed depends on the LLM agent to stop each experiment. If something goes wrong on the LLM side (it forgets, hallucinates or fails), the stop request never arrives and **the experiment keeps running without anyone knowing**, holding testbed resources. The template therefore arms a watchdog for every experiment.

9.2 The max-duration watchdog

When an experiment starts, `start_experiment()` arms a **watchdog**: a timer set to `MAX_EXPERIMENT_DURATION` seconds. Then one of two things happens:

Situation	What happens
Stop endpoint called in time	`stop_experiment()` cancels the watchdog, then calls `stop_procedure(exp_id, "stop request")`.
No stop request within `MAX_EXPERIMENT_DURATION`	The watchdog calls `stop_procedure(exp_id, "max duration reached")` and logs a warning.

Both paths end in the same `stop_procedure()`, so your stopping logic is written once. `MAX_EXPERIMENT_DURATION` is an **environment variable** (seconds, default 3600), set at deployment to the longest run your testbed should ever allow. The engine refuses to start if the value is not a positive number.

```
# shell
MAX_EXPERIMENT_DURATION=7200 python translation_engine.py

# Docker
docker run -e MAX_EXPERIMENT_DURATION=7200 ...

# systemd unit
Environment=MAX_EXPERIMENT_DURATION=7200
```

- Keep the `_start_watchdog(exp_id)` call in `start_experiment()`; implement the stop itself in `stop_procedure()`.
- The `reason` argument tells you which path stopped the experiment. The template treats a watchdog stop like a normal stop: the data collected so far goes through dataset, metadata and upload, and the final state follows the COMPLETED rule (section 6.1). If a forced stop makes your data unusable, report `FAILED` when `reason` is `"max duration reached"`.
- `stop_procedure()` removes the experiment from your state first, so a stop request arriving while the watchdog is stopping it receives `404 EXPERIMENT_NOT_FOUND` instead of stopping it twice.

10. Implementation checklist

1. Run the template unchanged and open `/docs` (section 2).
2. Define `Configuration` and `EXAMPLE_CONFIG` in `models.py`, keeping `extra="forbid"` and a `description` on every field.
3. Give every `config` field a type, a range or enum, and a description (section 8).

4. Set `TESTBED_NAME` in `testbed.py`.
5. Replace `_experiments` with your own experiment tracking, including `status_update_url`.
6. Implement `start_experiment()`: testbed checks, translation of `config`, start call, `TESTBED_UNAVAILABLE` / `TESTBED_ERROR` handling, background pipeline.
7. Implement `process_metadata()`: hand `metadata` to your metadata generation solution.
8. Implement `stop_experiment()`: `EXPERIMENT_NOT_FOUND`, stop call, final state.
9. Check the state rules: one final state per experiment, `COMPLETED` only after run + dataset + metadata + DS connector upload all succeeded, `FAILED` for any failure.
10. Implement `stop_procedure()`, keep the watchdog, and set `MAX_EXPERIMENT_DURATION` at deployment (section 9).
11. Make sure no blocking call is used in async functions (use `httpx.AsyncClient`, or `asyncio.to_thread()` for blocking libraries).
12. Test the full cycle with `extras/mock_framework.py`, including failure paths and invalid requests.
13. Confirm you have not modified `translation_engine.py`, `utils.py` or the common sections of `models.py`.